Modelling Tyre Degradation in Formula 1

using a Data-Driven Machine Learning Pipeline

ME228 Course Project — Indian Institute of Technology

Abstract

Tyre degradation is one of the most consequential yet frustratingly difficult variables to study in Formula 1 race strategy. The core challenge is that observed lap times are inherently noisy — a slow lap might reflect genuine rubber wear, but it might just as well be caused by traffic, a yellow flag, or a driver mistake. Isolating the true degradation signal from all of this noise is genuinely hard, and it is the central problem we have tried to tackle in this work.

Why does it matter? Because tyre wear has a direct, measurable effect on lap time, and understanding that relationship unlocks some interesting downstream applications — things like modelling friction through corners, figuring out how aggressively a driver can push before the tyres fall off a cliff, or helping a team decide how much lap time they are actually *losing* to tyre degradation versus, say, fuel load or track evolution. We built this pipeline specifically to try and quantify that relationship.

The pipeline has two stages. First, a Multi-Layer Perceptron (MLP) classifier is trained to identify *clean* laps — laps where a driver was in free air and not influenced by traffic, yellow flags, or anomalous driving events. Clean lap probabilities are then used as sample weights in a second stage where multiple learning models (Ridge, Gradient Boosting, Random Forest, and an MLP regressor) learn the relationship between tyre age, compound, track location, and lap time degradation. The pipeline is evaluated both in-sample and on entirely unseen races. Key findings include: (1) a simple gap-based classifier is sufficient to improve regression quality but has clear limitations; (2) telemetry-derived features produce significantly more reliable lap labels; (3) track-specific encoding is essential for generalisation; and (4) even with race encoding, cross-year generalisation remains a challenge, pointing towards the need for physics-grounded track features.

1. Introduction

In Formula 1, strategy decisions around pit-stop timing, tyre compound selection, and stint length are among the most performance-critical choices a team makes during a race. All of these decisions hinge on accurately predicting how quickly a tyre degrades. A tyre that degrades faster than expected forces an early pit stop, costing track position; one that holds longer can enable an undercut strategy that gains places.

Despite its importance, tyre degradation is notoriously hard to model. Observed lap times are a *noisy* signal — a slow lap might reflect genuine tyre wear, or it might reflect traffic, a yellow flag, driver error, or a DRS-induced gap change. Separating the *clean* degradation signal from this noise is the central challenge addressed in this work.

This project develops a two-stage pipeline:

1. **Clean lap classification.** An MLP is trained to assign each lap a probability of being a clean, representative tyre-wear lap.
2. **Degradation learning.** Multiple models — Ridge Regression, Gradient Boosting, Random Forest, and an MLP regressor — use these probabilities as sample weights to learn compound- and track-specific degradation patterns. The core signal we are trying to model is ΔT: how much slower a given lap is compared to the best lap the driver achieved on that same set of tyres during that stint. A positive and increasing ΔT over lap number is the fingerprint of tyre wear.

The report follows the pipeline step-by-step, from data collection through to out-of-sample evaluation, and compares all four model architectures under both versions of the clean-lap labels.

2. Data Collection

2.1. Source and Season Coverage

All data was collected using the FastF1 Python library [**?**], which provides access to the official Formula 1 timing feed. Two data loading configurations were used, driven by a practical constraint: telemetry data (240 Hz car channels) is significantly heavier to load and process than basic lap-level data, and loading it for all 22 races would have made training time prohibitive.

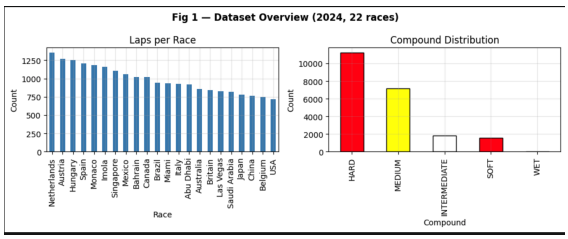
For **Stage 1 (V1) clean-lap classification and all regression models**, the full **2024 Formula 1 season** was used — all 22 races from Bahrain through Abu Dhabi. No telemetry was required here, so the full calendar was tractable to process.

For **Stage 1 (V2) telemetry-based classification**, we restricted training to six races where loading full 240 Hz telemetry was still manageable: **Bahrain, Saudi Arabia, Australia, Japan, China, and Miami**. Training on more races with telemetry would have been feasible given more time and compute, but for this project these six gave us sufficient coverage across different track types and conditions.

Out-of-sample generalisation for all models was tested on **Bahrain 2023** — an unseen race from a different year.

2.2. Available Fields

The lap-level dataset provided by `session.laps` includes: lap time, sector times, compound (SOFT / MEDIUM / HARD), tyre life, stint number, pit in/out times, track status codes, and the `IsAccurate` flag. Car telemetry (speed, throttle, brake, DRS, gear, and X/Y/Z position), sampled at 240 Hz, is available separately via `lap.get_telemetry()`.



[Fig. 1: Laps per race and compound distribution bar charts]

Figure 1: Dataset overview: number of laps per race and compound breakdown across the 2024 training races.

3. Data Cleaning and Preprocessing

Raw lap data contains a lot of entries that are not useful for modelling tyre wear. Before anything else, we need to strip out laps that are clearly unrepresentative. The following steps were applied in sequence.

3.1. Lap Filtering

- **Accuracy flag.** Only laps where `IsAccurate = True` were kept. This is FastF1's own flag for laps where the timing system had complete and reliable data.
- **Pit laps.** Any lap with a non-null `PitInTime` or `PitOutTime` was dropped. These laps include the pit stop time itself and are orders of magnitude slower — they have zero degradation signal.
- **Missing values.** Rows missing `LapTime`, `Compound`, `Driver`, or `Stint` were removed.
- **Sanity cap.** Laps above 200 seconds were dropped as likely formation laps, safety car periods, or timing errors.

3.2. Outlier Removal

Even after the above filters, some laps are anomalous. We apply a 3σ rule to remove them, but crucially this is done *per compound* independently. Applying a global threshold would be wrong here: SOFT laps are naturally faster than HARD laps, so a slow HARD lap might look fine globally but be a genuine outlier within the HARD distribution. Compound-wise filtering avoids this bias.

3.3. Stint-Level Feature Engineering

Once the data is clean, we compute a few key columns that everything else depends on:

- **LapInStint.** Position of each lap within its stint (1 = first lap after a pit stop or race start).
- **StintID.** A unique string identifier combining Race, Driver, and Stint number, used to keep the same driver's stint together during train/test splits.
- **RaceTime.** Cumulative race time per driver, computed from the cumulative sum of lap times.
- **GapAhead / GapBehind.** For each lap number, all cars are sorted by cumulative RaceTime and sequential differences are computed to give the gap to the immediately preceding and following car on track.

4. Degradation Signal

The regression target is ΔT, defined as the deviation of a lap time from the *best lap* within that stint:

$$\Delta T_i = t_i - \min_{j \in \text{stint}} t_j \tag{1}$$

This normalises away the absolute speed difference between compounds and drivers, leaving only the *relative degradation* signal — how much slower this lap is compared to the best the driver has managed on these tyres. A ΔT that increases as `LapInStint` grows is exactly what tyre wear looks like.

We do not apply any rolling average to ΔT before using it as the regression target. Smoothing across laps would mess up the clean and noisy laps, which is precisely the problem we are trying to solve with the classifier in Stage 1. Instead, we let the classifier's probability weights do the noise reduction at training time.

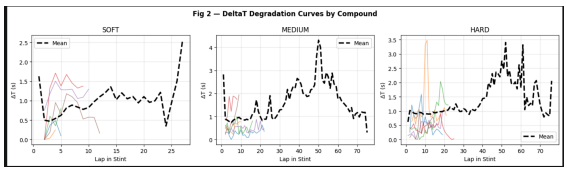


Figure 2: Degradation curves for individual stints. Each line is one driver stint. The upward trend captures tyre wear; the scatter reflects lap-to-lap noise.

5. Stage 1 — Clean Lap Classification

The goal of Stage 1 is to assign each lap a probability $p_{\text{clean}} \in [0, 1]$ reflecting how representative it is of true

tyre-wear behaviour. This probability is later used as a sample weight in Stage 2 so that the regression models pay more attention to clean laps and downweight noisy ones.

5.1. Version 1: Gap-Based Labelling

The simplest way to define a clean lap is to check whether the driver was in free air — not stuck behind a slower car and not being pressured from behind. A lap was labelled **clean** (label = 1) if both `GapAhead` and `GapBehind` exceeded 3.45 s (threshold 3 s with a 15% upper margin). It was labelled **noisy** (label = 0) if either gap fell below 2.55 s (3 s minus 15%). Laps falling between the two thresholds were treated as borderline and discarded — we did not want to train the classifier on ambiguous cases.

It is worth noting that in our midterm report, we used these gap thresholds directly as the final clean/noisy labels, without any classifier on top. The MLP layer was added in this final version to produce a *soft* probability rather than a hard binary label, which integrates more smoothly into the weighted regression.

This heuristic is simple but has a known failure mode: a driver can be in completely free air and still produce a slow lap due to a yellow flag, a DRS deployment, or an off-throttle moment. Section 6 addresses this.

5.2. Classifier Features

In addition to the raw gap values, several features were engineered to help the MLP pick up on anomalous driving conditions that gaps alone cannot detect:

Table 1: Features used in the MLP classifier.

Feature	Description
`GapAhead`	Gap to car ahead [s]
`GapBehind`	Gap to car behind [s]
`DeltaT_lag1`	ΔT of previous lap
`DeltaT_diff`	Change in ΔT between laps
`LapTime_rollingStd`	3-lap rolling std of lap time
`GapAhead_std`	3-lap rolling std of GapAhead
`LapInStint`	Lap position within stint
`LapFraction`	Relative position (0–1)
`IsFirstLap`	Binary flag: lap 1 of stint
`IsLastLap`	Binary flag: last lap of stint

5.3. MLP Architecture (V1)

The classifier is a feed-forward neural network implemented in PyTorch. It takes the 10 features above, passes them through two hidden layers with batch normalisation and dropout for regularisation, and outputs a single sigmoid-activated probability:

Layer 1: Linear(10 → 64) → BatchNorm → ReLU → Dropout(0.3)

Layer 2: Linear(64 → 32) → ReLU → Dropout(0.2)

Output: Linear(32 → 1) → Sigmoid

Binary cross-entropy loss was minimised using Adam (lr = 10^{-3}) over 100 epochs. The dataset was split *by* `StintID` rather than randomly by row — this ensures that a given driver's stint never appears in both training and test sets, which would otherwise cause data leakage.

5.4. MLP Evaluation (V1)

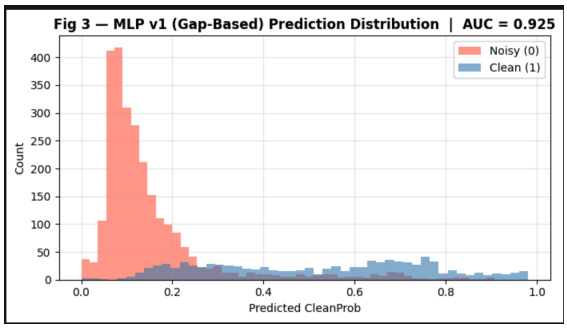


Figure 3: Distribution of predicted `CleanProb` for noisy (0) and clean (1) laps. Good separation indicates the classifier has learned a meaningful signal.

Table 2: MLP V1 classification metrics on the held-out test set.

6. Stage 1 Improved — Telemetry-Based Labelling

6.1. Motivation

A driver can be running in completely free air (large gaps ahead and behind) and still produce a slow lap because of a yellow flag, a DRS train in an adjacent section of track, or an on-throttle mistake. None of these situations are caught by gap-based labels. We need to look directly at what the car was doing during the lap.

6.2. Version 2: Telemetry Anomaly Score

For each lap, five features were extracted from the 240 Hz telemetry stream:

- **Speed_std** — standard deviation of speed within the lap. A high value suggests erratic or disrupted pace.
- **Throttle_std** — standard deviation of throttle position. Smooth clean laps have consistent throttle patterns.
- **FullThrottle_frac** — fraction of the lap spent above 95% throttle. This drops sharply under yellow flags or when a driver is managing tyres.
- **Brake_events** — number of distinct braking events.
- **Speed_drop_events** — number of samples where speed drops more than 15 km/h between consecutive measurements.

Each of these was z-scored *within its own stint* before combining. Z-scoring within the stint is important because it removes track-level and compound-level baselines — a lap that is anomalous relative to what *this driver on these tyres* was doing is the signal we want, not one that is anomalous relative to all cars at all tracks. A composite **TelemetryScore** was then computed as a weighted sum of absolute z-scores:

$$S_i = 0.5|z_{\text{Speed}}| + 0.5|z_{\text{Throttle}}| + 0.4(1 - \text{FullThrottle}) + 0.3|z_{\text{Brake}}| + 0.3|z_{\text{SpeedDrop}}| \tag{2}$$

Laps above the 60th percentile of S_i were labelled noisy; the rest were labelled clean. Additionally, any lap where `TrackStatus` $\neq$ `'1'` (i.e. not fully green flag) was hard-set to label 0, regardless of telemetry score.

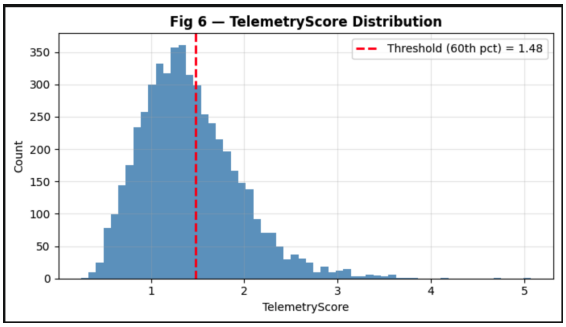


Figure 4: Distribution of `TelemetryScore` across all laps. The dashed line marks the classification threshold.

6.3. MLP V2: Architecture and Evaluation

The second MLP uses the same architecture as V1 (Section 5.3) but with an extended 15-feature input — the original 10 gap-based features plus the 5 telemetry channels described above. The same stint-based train/test split was applied.

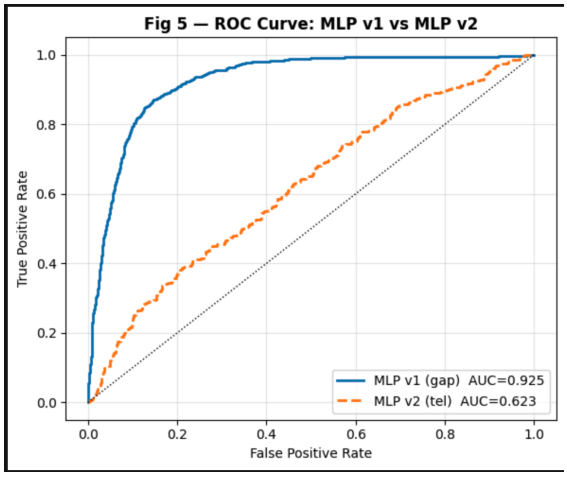


Figure 5: ROC curves for MLP V1 (gap-based) and MLP V2 (telemetry).

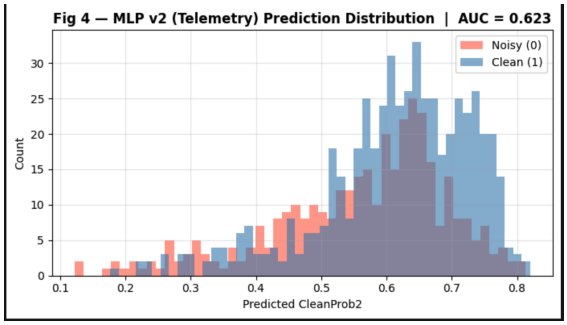


Figure 6: Predicted `CleanProb2` distributions for noisy and clean laps under the telemetry-based V2 classifier.

Table 3: MLP classifier comparison: V1 (gap-based) vs V2 (telemetry).

One thing worth noting here: we believe that AUC alone is not the most meaningful metric for comparing V1 and V2. The V1 labels are essentially derived from a single binary condition (gap above or below a threshold), so the MLP is learning an almost deterministic mapping — high AUC is almost expected. An MLP was not strictly necessary for V1, but we kept it to ensure a fair apples-to-apples comparison with V2. The V2 labels are genuinely probabilistic and multi-dimensional, so an MLP is much more appropriate there, and its AUC is arguably the more meaningful number of the two.

7. Stage 2 — Learning Tyre Degradation

7.1. Feature Construction

The regression target is `DeltaT` (raw, not smoothed). Four model variants were trained for each version of the classifier (V1 and V2), giving 8 total comparisons. For all models, the input features are:

$$\mathbf{x} = \left[\text{LapInStint},\ \text{LapInStint}^2,\ \mathbf{c}_{\text{compound}},\ \mathbf{c}_{\text{race}}\right] \quad (3)$$

where $\mathbf{c}_{\text{compound}}$ and $\mathbf{c}_{\text{race}}$ are one-hot encodings of tyre compound and race location respectively. The sample weight for each training observation is set to $w_i = p_{\text{clean},i}$ from the relevant MLP, so cleaner laps contribute more to the loss.

The quadratic $\texttt{LapInStint}^2$ term is included because tyre degradation is known to accelerate non-linearly with tyre age, particularly for the SOFT compound.

7.2. Model Architectures

Ridge Regression. A linear baseline with L2 regularisation ($\alpha = 1.0$). Simple and interpretable — effectively fits a compound- and race-specific quadratic degradation curve.

Gradient Boosting (GB). An ensemble of 200 shallow decision trees (depth 3, learning rate 0.05). Can capture non-linear interactions that Ridge cannot.

Random Forest (RF). Bagged ensemble of 200 trees with max depth 8. More variance-tolerant than GB; less sensitive to hyperparameter tuning.

MLP Regressor. A PyTorch feed-forward network: Linear($d \to 64$) $\to$ BatchNorm $\to$ ReLU $\to$ Dropout(0.2) $\to$ Linear($64 \to 32$) $\to$ ReLU $\to$ Linear($32 \to 1$). Trained with weighted MSE loss (weights = CleanProb).

7.3. Training and Evaluation Protocol

All four models were trained on the full 2024 dataset. Out-of-sample evaluation was conducted on Bahrain 2023. For each model, three metrics are reported:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}, \quad \text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \quad (4)$$

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (5)$$

8. Results

8.1. Learned Degradation Curves

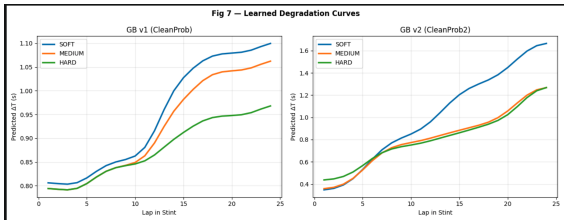


Figure 7: Predicted degradation curves from Gradient Boosting v1 and v2. Display-only smoothing ($\sigma = 1.5$) applied for readability.

8.2. In-Sample Performance

Table 4: In-sample regression results (2024 training races, all 8 models).

Model	RMSE	MAE	R^2
Ridge (V1)	*1.233*	*0.719*	*0.253*
GB (V1)	*1.09*	*0.651*	*0.411*
RF (V1)	*1.042*	*0.633*	*0.466*
MLP Reg (V1)	*1.20*	*0.694*	*0.29*
Ridge (V2)	*0.65*	*0.471*	*0.197*
GB (V2)	*0.582*	*0.411*	*0.362*
RF (V2)	*0.566*	*0.400*	*0.395*
MLP Reg (V2)	*0.610*	*0.433*	*0.299*

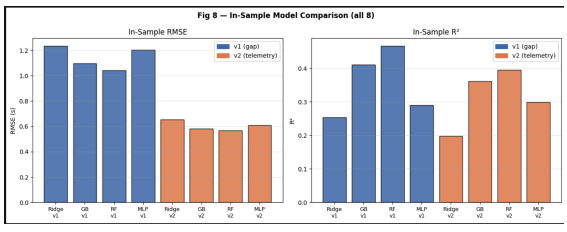


Figure 8: In-sample RMSE (left) and R^2 (right) for all 8 model variants. Blue = V1 (gap-based weights), orange = V2 (telemetry weights).

8.3. Out-of-Sample Generalisation

Table 5: Out-of-sample results on Bahrain 2023 (unseen year).

Model	RMSE	MAE	R^2
Ridge (V1)	0.621	0.502	-0.196
GB (V1)	0.554	0.438	0.45
RF (V1)	0.571	0.449	-0.011
MLP Reg (V1)	0.623	0.507	-0.206
Ridge (V2)	0.560	0.418	0.027
GB (V2)	0.510	0.376	0.192
RF (V2)	0.509	0.374	0.195
MLP Reg (V2)	0.540	0.402	0.093

[Fig. 9: Bar charts — OOS RMSE and R^2 for all 8 models, Bahrain 2023]

Figure 9: Out-of-sample RMSE (left) and R^2 (right) for all 8 model variants on Bahrain 2023.

8.4. Per-Driver Stint Prediction: Perez, Soft, Bahrain 2023

A particularly informative visualisation is to look at individual driver stints and see how the eight models track the actual ΔT trajectory. Figure 10 shows Sergio Perez's soft tyre stint in Bahrain 2023 (out-of-sample), with all eight model predictions overlaid.

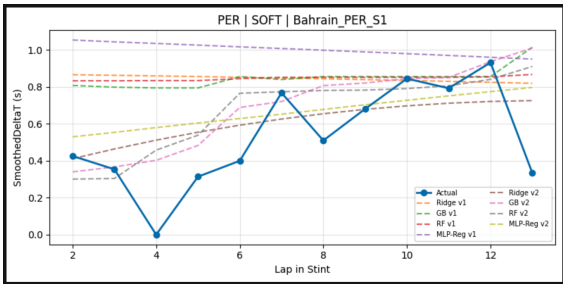


Figure 10: Actual ΔT vs predicted for all 8 models on a Perez SOFT stint in Bahrain 2023. Note how GB and RF in the V2 configuration track the degradation trend most closely.

This is a good example of the broader pattern we see across stints: gradient boosting and random forest under the V2 (telemetry-weighted) configuration tend to track the overall degradation trend more faithfully than their V1 counterparts.The MLP regressor is more variable and does not fit very well. It probably requires better tuning or a different structure

9. Discussion

9.1. Effect of Track Encoding

The most impactful single improvement across all models was the addition of `Race` as a one-hot encoded categorical feature. Without it, a model trained on multiple 2024 races and tested on an unseen one produced strongly negative R^2 values (-2 to -3) This makes intuitive sense: Bahrain's abrasive tarmac produces much faster tyre wear than Monaco's smooth, slow surface, and a compound-only model has no way to account for this.

Adding race encoding gives each track a specific bias term, which substantially improves both in-sample fit and generalisation to unseen laps of the *same* race.

9.2. Gap-Based vs Telemetry-Based Labelling

The gap-based (V1) labels are cheap to compute and produce a reasonable classifier. But they conflate being in free air with actually driving at representative pace. Telemetry-based (V2) labels directly measure driving behaviour and catch anomalies that gap information cannot.

The key insight in V2 is the intra-stint z-scoring: a lap is flagged as noisy if it is anomalous relative to *what the driver was doing on these tyres*, not relative to some global average. This makes the label more sensitive to actual on-track disruptions while being robust to compound- and track-level differences in absolute pace.

9.3. Limitations

- **Cross-year generalisation.** Even with race encoding, testing on 2023 data from a 2024-trained model showed meaningful drops. Car performance, regulations, and compound allocations changed between seasons.
- **Race encoding is a proxy.** One-hot encoding captures track identity but not track physics (surface abrasion, corner geometry, ambient temperature). True generalisation requires explicit track features which we could model (We did not have enough time to develop a proper mathematical model for this).
- **Telemetry cost.** Extracting telemetry for all laps across 22 races requires loading 240 Hz data for ∼800 stints, which is slow — hence our restriction to 6 races for V2.

10. Conclusion

This project demonstrates a complete pipeline for modelling Formula 1 tyre degradation using only publicly available data. The main contributions are:

1. A **two way lap filtering approach**, first using simple gap thresholds (V1) and then telemetry-derived anomaly scores (V2), both converted into soft clean probabilities by an MLP classifier.

2. A **systematic comparison of four regression architectures** (Ridge, Gradient Boosting, Random Forest, MLP regressor) under both labelling strategies, evaluated both in-sample and on an unseen race-year combination.

3. The **identification of track encoding** as an important factor for generalisation, and a clear characterisation of where categorical race labels fall short (We do believe that there are other factors that will inprove generalisation which we could study given more time).

The central finding is that *data quality matters more than model complexity*: the largest gains came from better labelling (V2 over V1) and from adding track context (race encoding), not from switching from Ridge to Gradient Boosting.

Looking at the results across models, Gradient Boosting and Random Forest under the V2 (telemetry-weighted) configuration consistently outperform their V1 counterparts and outperform Ridge and the MLP regressor in most metrics — both in-sample and out-of-sample. This is visible clearly in the per-stint plots (e.g. Figure 10), where GB-V2 and RF-V2 track the actual degradation trajectory most faithfully, while Ridge-V1 is too flat and the MLP regressor is more erratic. V2 models generally produce lower RMSE and higher R^2 than V1 counterparts with the same architecture, confirming that telemetry-derived weights are a genuine improvement over gap-based ones.

Future directions include replacing the categorical race encoding with physically meaningful track features (lap-time distribution, abrasion index, corner speed profile) and extending the pipeline to include fuel load correction, which is currently a confounding variable in early-stint laps.